

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. : 42	Name: Aniket Tiwari
Class : SYIT	Batch : 00
Date of Assignment : 22-07-2026	Date/Time of Submission : 22-07-2026

Binary Search Tree: Write a program to:

- a. Create a binary search tree.

CODE:

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG

#Test
print("root.right.left.data:", root.right.left.data)
print("Aniket")
```

OUTPUT:

```
----- RESTART: P
root.right.left.data: E
Aniket
```

b. Insert nodes into a binary search tree.

CODE:

```
class TreeNode:
```

```
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
```

```
root.left = nodeA
root.right = nodeB
```

```
nodeA.left = nodeC
nodeA.right = nodeD
```

```
nodeB.left = nodeE
nodeB.right = nodeF
```

```
nodeF.left = nodeG
#Test
print("root.right.left.data:", root.right.left.data)
```

```
def insert(root, data):
    if root is None:
        return TreeNode(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
```

```

return root

new_node = input("Enter element to insert: ")
root = insert(root, new_node)

print("Node inserted successfully.")
print("Aniket")

```

OUTPUT:

```

===== RESTART: D:/An
root.right.left.data: E
Enter element to insert: A
Node inserted successfully.
Aniket

```

c. Search for a node in a binary search tree.

CODE:

```

class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

```

```

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

```

```

root.left = nodeA

```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
```

```
#Test
```

```
print("root.right.left.data:", root.right.left.data)
```

```
def search(root, key):
```

```
    if root is None:
```

```
        return False
```

```
    print("Checking:", root.data)
```

```
    if root.data == key:
```

```
        return True
```

```
    return search(root.left, key) or search(root.right, key)
```

```
key = input("Enter element to search: ")
```

```
# Search
```

```
if search(root, key):
```

```
    print("Element Found")
```

else:

```
    print("Element Not Found")
```

```
print("Aniket")
```

OUTPUT:

```
===== RESTART: D:/A
root.right.left.data: E
Enter element to search: A
Checking: R
Checking: A
Element Found
Aniket
```